MC-RayTracer Environment Lighting: Complete Technical Report

Codebase Audit (CPU+GPU Paths)

Generated: 2026-04-27

Contents

1 Scope

This report documents **all environment-lighting related code paths** in:

- `include/environment.h`
- `include/EnvironmentLight.h` + `src/EnvironmentLight.cpp`
- `include/ILight.h`
- `include/LightFactory.h` + `src/LightFactory.cpp`
- `include/LightJsonParser.h` + `src/LightJsonParser.cpp`
- `src/EnvironmentLightRegistration.cpp`
- `include/scene.h`
- `include/texture.h`
- `include/query.h` + `include/query.cu`
- `include/shader.h`
- `src/main.cu`

It explains:

- where environment light comes from,
- what each file and each relevant function does,
- what runtime behavior you get today,
- what is affected in CPU and GPU modes.

2 Big-Picture Architecture

2.1 Two parallel environment-light systems currently coexist

System A (active render path):

- Uses `HDRTextureData` in `texture.h`.
- Evaluates background and direct env NEE in `query.h`.
- Builds env importance CDF in `main.cu` via `BuildEnvImportance(...)`.
- Feeds `EnvImportanceData` into CPU `TraceRayIterative(...)`.

System B (polymorphic OOP path):

- `ILight` interface, `EnvironmentLight` implementation.

- Factory/registry/parsing via `LightFactory` and `LightJsonParser`.
- Registered by `EnvironmentLightRegistration.cpp`.
- Currently used for runtime parse/validation wiring in `main.cu`, but not as the actual sampler in the integrator.

2.2 Energy contributors in final image

1. **Direct finite lights**: `ShadeDirect(...)` over `scene.lights` (point + directional).
2. **Emissive triangles**: explicit NEE + MIS logic in `TraceRayIterative(...)`.
3. **Environment (infinite light)**:
   - Miss-path contribution.
   - Explicit env-light NEE strategy (section 7b in `TraceRayIterative`).

3 Control Flow: Where Environment Light Comes From

3.1 JSON sources

Environment-related parameters are read from:

- top-level `environment` block (`scene.h::parse_environment_block`),
- top-level `lights[]` entries with `type == "environment_light"` (`scene.h` and `LightJsonParser.cpp`).

3.2 Precedence in current code

- `environment.hdri_path` sets `scene.sky_hdri_path` first.
- In `lights[]`, `environment_light` can override:
  - `hdri_path` (if enabled),
  - `intensity_scale -> scene.environment.map_intensity`,
  - `tint -> scene.environment.tint`,
  - `rotation_euler_deg[2] -> scene.environment.map_rotation_deg`.
- If **no directional light exists**, fallback directional sun is synthesized from: `environment.sun_dir/sun_tint/`

4 Header-by-Header Responsibilities

4.1 `include/ILight.h`

Defines stable ABI for polymorphic lights:

- `Sample(normal, u, pdf_out)`: sample incoming direction.
- `Pdf(wi)`: solid-angle PDF.

- `Eval(wi)`: emitted radiance.
- `IsDelta()`: marks singular lights.

4.2 `include/EnvironmentLight.h`

Declares environment light implementation:

- `EnvironmentLightParams`: enabled flag, HDRI path, intensity/tint, importance resolution, Euler rotation.
- `EnvironmentLight` class:
  - public MIS API from `ILight`,
  - private map loading, rotation matrices, and 2D importance distribution.

4.3 `include/LightFactory.h`

String-to-constructor registry for `ILight` subclasses:

- `Register(type, creator)`
- `Create(type, params)`

4.4 `include/LightJsonParser.h`

Declares non-throwing parser:

- `ParseTopLevelLights(root, out_lights)` and summary counts.

4.5 `include/environment.h`

Procedural/latlong environment model and helpers:

- defines `Environment` struct (sun, sky, atmosphere, tone-map controls, debug modes),
- defines environment radiance evaluation and physical sky math,
- defines final image post-process helper `FinalizeImagePixel(...)`.

4.6 `include/texture.h`

Defines texture data structures used by environment pipeline:

- `HDRTextureData`: float HDR map view + tint + intensity + azimuth rotation.
- `EnvImportanceData`: GPU-compatible CDF/PMF view for importance sampling.
- `LoadHDRTexture(...)` and `LoadTexture(...)`.

4.7 `include/scene.h`

Scene parsing and fallback logic:

- parses `environment` block into `Scene.environment`,
- parses `lights[]`,
- handles `environment_light` special entry,
- creates fallback directional sun if no directional light exists and environment enabled.

4.8 `include/query.h`

Integrator core:

- `sampleHDRI(...)`
- env MIS helpers (`env_light_eval/pdf/sample_dir` + CDF sampler)
- `TraceRayIterative(...)` miss-path env contribution and env NEE strategy.

4.9 `include/query.cu`

Render wrapper:

- GPU kernel and CPU loop call `TraceRayIterative(...)`,
- passes `EnvImportanceData*` only through CPU path today.

4.10 `include/shader.h`

Finite direct light shading path:

- `ShadeDirect(...)` for point/directional `Light` array,
- directional sunlight shadow behavior in `IsInShadow(...)`.

5 Function-by-Function Catalog

5.1 `include/environment.h`: full environment utility and sky model

Enum/string helpers

- `EnvironmentPresetToString(...)`: preset enum to token.
- `EnvironmentPresetFromString(...)`: token to preset enum.

Math primitives

- `env_clampf`: clamp scalar.
- `env_lerp_f`: scalar lerp.

- `env_lerp`: vector lerp.
- `env_smoothstep`: smooth interpolation.
- `env_fract`: fractional part.
- `env_safe_normalize`: robust normalization with fallback.
- `env_hash3`: deterministic noise hash (stars/twinkle support).

Direction-angle conversion

- `DirectionFromElevationAzimuthDeg`: elevation/azimuth to world direction.
- `ElevationAzimuthFromDirectionDeg`: inverse mapping.

Preset application

- `ApplyEnvironmentPreset`: fills environment defaults by preset; custom exits early preserving JSON.

Lat-long map sampling

- `env_sample_latlong_texel`: fetches nearest texel from 8-bit map.
- `EvaluateLatLongEnvironment`: bilinear sampled latlong evaluation with yaw rotation and intensity.

Atmosphere/physics helpers

- `env_earth_radius_m`, `env_beta_R_sea_level`, `env_beta_M_sea_level`,
- `env_turbidity_to_mie_factor`,
- `env_ray_sphere`,
- `env_phase_rayleigh`, `env_phase_mie_hg`.

Scattering and celestial terms

- `env_view_transmittance`: Beer-Lambert transmittance through atmosphere.
- `env_single_scattering`: Nishita-style single scattering (Rayleigh + Mie).
- `env_sun_disk`: physically motivated bright solar disc term.
- `env_legacy_sky`: artistic non-physical sky fallback.
- `env_night_sky`: moon + stars model.

Output-domain helpers

- `env_aces_channel` and `env_tonemap_aces`: display tonemapping.
- `EvaluateEnvironment`: master linear radiance environment function.
- `FinalizeImagePixel`: exposure + optional ACES applied once at end.
- `EnvironmentDebugPrint`: debug dump of resolved parameters.

5.2 src/EnvironmentLight.cpp: OOP environment light implementation

Internal helpers (anonymous namespace)

- clamp01: safe [0,1] clamp.
- luminance709: BT.709 luminance.
- mat_mul, mat_transpose, mat_mul_vec: 3x3 rotation math.
- dir_from_uv, uv_from_dir: latlong direction/UV conversion.
- sample_cdf: CDF inversion for importance sampling.

Class methods

- EnvironmentLight::EnvironmentLight: copies params, sets env fields, builds rotation + map + PMF/CDF.
- HasMap: verifies map availability.
- LoadLatLongMap: loads map via stbi_load (8-bit RGB), stores as TextureData.
- BuildRotationMatrices: Euler XYZ to world/env transform matrices.
- EnvToWorld / WorldToEnv: apply transform.
- EvalLatLong: evaluate map-backed environment radiance through EvaluateEnvironment.
- BuildImportanceDistribution: computes row/column CDF and PMF with sin(theta) weighting.
- Sample: sampled direction from PMF/CDF or uniform-sphere fallback.
- Pdf: solid-angle PDF from PMF and pixel differential solid-angle.
- Eval: emitted radiance for world direction.
- IsDelta: returns false.

5.3 include/LightFactory.h + src/LightFactory.cpp

- Register(type, creator): thread-safe insertion/replacement in registry map.
- Create(type, params): thread-safe lookup and instance creation.

5.4 include/LightJsonParser.h + src/LightJsonParser.cpp

Utility readers

- read_number, read_bool, read_string, read_vec3, read_res2.

Main parsing

- parse_environment_light_params: validates and fills EnvironmentLightParams.
- ParseTopLevelLights: walks lights[], creates environment_light via factory, returns loaded/skipped/failed counters.

5.5 src/EnvironmentLightRegistration.cpp

- static kRegisteredEnvironmentLight: registers type key "environment_light" with factory lambda at startup.

5.6 include/texture.h environment-related functions

- HDRTextureData::sample(u,v): bilinear float HDR sample.
- LoadHDRTexture(path): stbi_loadf loader for linear HDR.
- EnvImportanceData: POD view for PMF/CDF used by env sampling.

5.7 include/scene.h environment-related functions

- parse_environment_block: loads all environment keys.
- parse_one_light: parses point/area/directional finite lights.
- lights[] environment_light branch: maps env-light JSON into scene.environment and scene.sky_hdri_path.
- fallback sun block: if no directional light and environment.enabled, synthesize directional sun.
- LoadSceneFromFile: parses JSON text and dispatches to parse_scene.

5.8 include/query.h environment-related functions

- sampleHDRI: world direction to equirectangular sample.
- env_light_eval: rotated env evaluation with tint * intensity.
- env_cdf_sample: device-safe binary CDF inversion.
- env_light_pdf: PMF-to-solid-angle PDF conversion, fallback 1/(4pi).
- env_light_sample_dir: 2-stage CDF inversion (row then column), fallback uniform sphere.
- TraceRayIterative(..., hdri, env_importance):
  - miss path: adds environment radiance, MIS-weights BSDF strategy in nee_mode==2,
  - section 7b: explicit env-light NEE sampling with power heuristic against BRDF pdf.

5.9 include/query.cu environment-related functions

- renderBatchCUDA: GPU kernel calls TraceRayIterative (without env importance pointer today).
- render(...) host wrapper:
  - CPU branch passes env_importance to TraceRayIterative,
  - GPU branch does not pass env-importance data into kernel in current implementation.

5.10 `include/shader.h` environment-light interaction

- `IsInShadow`: directional-light shadow rays (infinite distance test).
- `ShadeDirect`: accumulates finite `scene.lights` direct contribution.

5.11 `src/main.cu` environment pipeline functions

- `BuildEnvImportance(const HDRTextureData&, float az_rot)`:
  - computes luminance-weighted PMF and CDFs over HDR map,
  - uses `sin(theta)` Jacobian in sampleHDRI UV convention,
  - writes `EnvImportanceData` view.
- scene load path stores `scene_json_path` then calls:
  - `ParseTopLevelLights` (runtime parse/validation and factory wiring),
  - `BuildEnvImportance` for CPU render.
- HDRI setup:
  - sets `tint`, `intensity`, `az_rot` into HDR data structs.

6 How It Works End-to-End at Runtime

6.1 Step 1: Parse

1. JSON parsed by `SceneIO::LoadSceneFromFile`.
2. `environment` block fills `Scene.environment`.
3. `lights[]` parsed:
   - directional/point/area become finite `Light` entries,
   - `environment_light` updates HDRI path/tint/intensity/rotation controls.
4. If no directional exists and environment enabled, fallback sun light is added.

6.2 Step 2: Prepare data in `main.cu`

1. Load float HDRI via `LoadHDRTexture`.
2. Copy to:
   - GPU struct (`d_hdri`): includes tint/intensity/az_rot,
   - CPU struct (`h_hdri_data`): includes tint/intensity/az_rot.
3. Build CPU importance map CDF via `BuildEnvImportance`.
4. Pass `EnvImportanceData*` to CPU render path.

6.3 Step 3: Integrate in `TraceRayIterative`

When a ray misses:

- evaluate env radiance `Le = env_light_eval(hdri, dir)`,
- in MIS mode for bounced non-delta surface events:
  - compute `pdf_env = env_light_pdf(...)`,
  - apply BSDF-strategy weight `w_bsdf = power_heuristic(prev_pdf, pdf_env)`,
  - accumulate `throughput * Le * w_bsdf`.
- otherwise accumulate full `throughput * Le`.

At each non-miss surface hit (env NEE strategy 7b, MIS mode):

- sample `wi` from env CDF (`env_light_sample_dir`),
- shadow-test to infinity,
- evaluate `Le` and BRDF `f`,
- get BRDF pdf `pdf_B`,
- MIS weight `w_L = power_heuristic(pdf_L, pdf_B)`,
- accumulate `throughput * Le * f * (NdotL/pdf_L) * w_L`.

7 What It Affects

- **Background color**: miss rays now use rotated/tinted/intensity-scaled HDRI.
- **Direct illumination quality**: explicit env NEE reduces variance vs pure BSDF-only escape.
- **Sun behavior**: if no explicit directional, fallback sun follows `environment.sun_*`.
- **Scene tone/warmth**: `environment_light.tint` and `intensity_scale` directly modulate env radiance.

8 Current Limitations / Notes

- GPU kernel path currently does not consume uploaded env importance CDF pointers in `renderBatchCUDA` path; CPU path does.
- `environment_light.enabled=false` only disables the light-entry override branch; if `environment.hdri_path` is set, HDRI may still be active.
- `rotation_euler_deg` is effectively yaw-only in scene parser (`[2]` used as azimuth), while OOP `EnvironmentLight` can build full Euler matrices.

- OOP `EnvironmentLight` is runtime-parsed and registered, but integrator uses the float-HDR/CDF path for actual sampling.
- In `query.cu` AOV miss assignment still uses raw `sampleHDRI` in one path (without env helper modifiers).

9 Practical Mental Model

1. **Scene parser decides controls**: sun params, HDRI path, env tint/intensity/rotation.
2. **main.cu materializes resources**: loads HDR, builds PMF/CDF, prepares render inputs.
3. **query.h performs physics estimator**: combines BSDF strategy and env-light strategy with MIS.
4. **shader.h handles finite lights**: directional sun fallback and point lights remain in direct-light loop.

10 Conclusion

You currently have a **hybrid environment-lighting system**:

- robust, physically-motivated environment model (`environment.h`),
- integrated CPU env MIS path with rotation/tint/intensity and CDF importance sampling,
- finite-light path for sun/point lights,
- OOP light architecture compiled and runtime-validated but not yet the active integrator sampling backend.

If you want a single unified architecture, the next step is to choose one of:

- make OOP `EnvironmentLight` the sole sampler/evaluator in render path, or
- keep current float-HDR path as canonical and use OOP path only for tooling/authoring.